MICROSOFT CERTIFICATION PREP

AI-103: Developing AI Apps and Agents on Azure

The Complete Visual Study Guide

120 Slides · Every Domain, Every Foundry Concept, Every Mnemonic You Need to Pass

WELCOME

How to Use This Guide

1. **Read actively.** Each concept slide pairs a Foundry capability with an exam-relevant implementation scenario — picture yourself building it.

2. **Trust the mnemonics.** Gold “Memory Booster” slides appear at regular intervals. They compress whole checklists into one recallable phrase.

3. **Practice as you go.** Practice Check slides simulate real exam wording — scenario-based, with real trade-offs. Reason it out before checking the answer.

4. **Follow the five domains.** Everything maps to Microsoft's official blueprint: Plan & Manage, Generative & Agentic Solutions, Computer Vision, Text Analysis, and Information Extraction.

5. **This exam is hands-on.** AI-103 expects real Python/Foundry SDK familiarity — pair this guide with actual time in the Foundry portal.

EXAM AT A GLANCE

Know the Test Before You Take It

40–60

Questions

Mix of multiple-choice, multiple-response, and scenario items

100

Minutes

Timed exam window at a Pearson VUE testing center

700

Passing Score

Scaled score out of 1000; no partial credit per topic

5

Skill Domains

Plan & Manage, Generative/Agentic, Vision, Text, Information Extraction

AI-103 replaces AI-102 (retiring June 30, 2026) — same associate level, rebuilt around Microsoft Foundry, generative AI, and agents.

YOUR ROADMAP

A Simple 4-Step Study Plan

1

Learn the Platform

Work through Domain 1: choosing, deploying, securing, and governing Foundry solutions.

2

Learn to Build

Work through Domain 2: generative apps, agents, and optimization — the largest domain by far.

3

Learn the Modalities

Work through Domains 3–5: computer vision, text/speech, and information extraction.

4

Simulate the Exam

Answer every Practice Check slide cold, then review the explanations and drill the mnemonics.

Because AI-103 is hands-on, budget real time inside the Foundry portal alongside this guide — not just reading time.

THE BLUEPRINT

What Microsoft Actually Tests

1. **Domain 1 — Plan and Manage an Azure AI Solution (25–30%):** Choosing models and Foundry services, setting up infrastructure and deployments, managing/monitoring/securing systems, and implementing responsible AI.

2. **Domain 2 — Implement Generative AI and Agentic Solutions (30–35%):** Building generative applications, building agents with Foundry, and optimizing/operationalizing generative AI systems. The single largest domain.

3. **Domain 3 — Implement Computer Vision Solutions (10–15%):** Image and video generation, editing workflows, and visual content safety.

4. **Domain 4 — Implement Text Analysis Solutions (10–15%):** LLM-powered text analysis and speech solutions for agentic interactions.

5. **Domain 5 — Implement Information Extraction Solutions (10–15%):** Retrieval and grounding pipelines — ingestion, indexing, search, and RAG.

Notice the center of gravity: roughly two-thirds of the exam sits in Domains 1 and 2 — planning and building generative/agentic solutions.

DOMAIN ONE

Plan and Manage an Azure AI Solution

Choosing the right models and Foundry services, setting up infrastructure and deployments, managing and securing systems, and implementing responsible AI.

Domain Weight: 25–30% of the exam

THE ROLE

What Does an Azure AI App & Agent Developer Do?

1. **The shift from AI-102:** AI-102 tested designing and implementing Azure AI services; AI-103 tests building generative AI apps and agents on Microsoft Foundry.

2. **Who you work with:** Business stakeholders, solution architects, data scientists, DevOps engineers, and cloud security engineers.

3. **What you actually build:** Generative applications, retrieval-augmented pipelines, and autonomous or semiautonomous agents — deployed, monitored, and governed in production.

4. **Exam framing:** Expect scenario questions with real trade-offs — cost vs. latency, security vs. convenience, autonomy vs. control.

Choosing the Right Model for the Task

1. **Large Language Model (LLM):** Broad reasoning and language capability — best for complex, open-ended generative tasks.

2. **Small Language Model (SLM):** Lower cost and latency, can run at the edge — best for narrow, well-defined tasks at scale.

3. **Multimodal Model:** Processes and/or generates more than one content type — text plus images, audio, or video together.

4. **Foundry Tools:** Pre-built capabilities (search, content understanding, code interpreter) you attach to a model rather than build yourself.

MODEL SELECTION

Four Categories of Model to Choose From

1 LLM

General-purpose, broad reasoning — chat, complex drafting, multistep reasoning tasks.

2 SLM

Fast, cheap, narrow — classification, extraction, and simple structured tasks at high volume.

3 Code Model

Specialized for generating, explaining, or reviewing source code.

4 Multimodal Model

Understands or generates combinations of text, image, audio, and video.

CHOOSING SERVICES

Matching Foundry Services to the Task

1. **Generative tasks:** Model deployments and playgrounds for drafting, summarizing, and conversational generation.

2. **Grounding:** Retrieval services (like Azure AI Search) that supply real data so generated answers are accurate.

3. **Vector search:** Similarity-based retrieval over embeddings — the backbone of most RAG grounding pipelines.

4. **Agent workflows & multimodal processing:** Agent Service for orchestrating tools and steps; Content Understanding for multimodal extraction.

Choosing a Retrieval & Indexing Method

1. **Keyword (lexical) search:** Matches exact terms — fast and precise, but misses paraphrased or conceptually related content.
2. **Vector (semantic) search:** Matches by meaning using embeddings — finds conceptually similar content even with different wording.
3. **Hybrid search:** Combines keyword and vector search, then re-ranks — the recommended default for most RAG solutions.
4. **Exam cue:** “Users search with different words than the source documents use” points to vector or hybrid search, not keyword-only.

CHOOSING SERVICES

Memory, Tool & Knowledge Integration for Agents

1. **Memory:** Lets an agent retain context across turns or sessions — short-term (conversation) or long-term (persisted facts).

2. **Tools:** Functions, APIs, or built-in capabilities (code interpreter, file search) an agent can invoke to take action.

3. **Knowledge:** Connected data sources — search indexes, files — an agent grounds its answers in.

4. **Exam cue:** A scenario needing an agent to “remember what the user said earlier” is a memory-integration question, not a tools question.

PRACTICE CHECK 1

A team is building a RAG solution where users often phrase questions very differently from the wording in the source documents. Which retrieval method should they configure?

A Keyword (lexical) search only

B Hybrid search combining vector and keyword retrieval ✓ CORRECT

C A larger context window with no retrieval

D Increasing the model's temperature parameter

Why: Hybrid search captures both exact-term matches and semantically similar content, making it the most robust default when query wording diverges from document wording.

SETTING UP FOUNDRY

Designing Azure Infrastructure for AI Apps & Agents

1. **Foundry project as the hub:** A project groups the models, agents, connections, and deployments for one solution together.

2. **Resource organization:** Decide how projects map to resource groups and subscriptions — typically by team, environment, or workload.

3. **Networking decisions:** Public endpoint vs. private endpoint access shapes both security posture and latency.

4. **Exam cue:** Infrastructure questions test whether you can match a compliance or isolation requirement to the right project/network layout.

GOING LIVE

Choosing a Deployment Option

Serverless (Pay-as-you-go)

- No infrastructure to provision
- Billed per token/request consumed
- Fastest way to get an endpoint live
- Best for variable or unpredictable traffic

Managed Compute (Dedicated)

- Reserved GPU/CPU capacity you provision
- Predictable billing and predictable latency
- More control over isolation and scaling
- Best for steady, high-volume production load

SETTING UP FOUNDRY

Configuring Model & Agent Deployments

1. **Deployment name & version:** Every deployment gets a stable name your app targets, decoupled from the underlying model version.

2. **Capacity settings:** Tokens-per-minute and requests-per-minute limits shape both cost ceiling and throughput.

3. **Content filter policy:** Attached per deployment — controls what's blocked before it ever reaches your application code.

4. **Agent configuration:** Beyond a model deployment, an agent also needs its instructions, tools, and knowledge sources configured.

CI/CD for Foundry Projects

1. **Why it matters:** Prompt templates, agent definitions, and evaluation configs are code — they deserve the same rigor as application code.
2. **What gets automated:** Deploying new model versions, running evaluation suites, and promoting agent configurations across environments.
3. **Environment strategy:** Dev → test → production Foundry projects, each with its own deployments and guardrail settings.
4. **Exam cue:** "Automatically re-run evaluations whenever a prompt changes" is a CI/CD-pipeline scenario.

AUTOMATING RELEASES

A Foundry CI/CD Pipeline

1

Commit

A developer updates a prompt, agent config, or code in source control.

2

Build & Evaluate

The pipeline runs automated evaluation suites against the change.

3

Approve

Results are reviewed against quality and safety thresholds before promotion.

4

Deploy

The updated model, prompt, or agent is promoted to the next environment.

Treating prompts and agent configs as versioned artifacts is what makes safe, repeatable Foundry releases possible.

MEMORY BOOSTER

Recall the Foundry Setup Chain

“Design → Deploy → Configure → Connect”

D	**Design**	Lay out the Azure infrastructure — projects, networking, resource organization.
D	**Deploy**	Choose serverless or managed compute for the model or agent.
C	**Configure**	Set deployment names, capacity, content filters, and agent settings.
C	**Connect**	Wire it into CI/CD so changes are tested and promoted automatically.

Every Foundry setup question fits somewhere on this chain — find the step, find the answer.

PRACTICE CHECK 2

A production support agent needs predictable latency and cost at a steady, high daily request volume. Which deployment option best fits?

A Serverless pay-as-you-go

B Managed compute with reserved capacity ✓ CORRECT

C No deployment — call the base model directly

D A local, unmanaged container only

Why: Managed compute with reserved capacity gives predictable latency and billing, which is exactly what steady, high-volume production workloads need.

MANAGE & MONITOR

Managing Quotas, Scaling, Rate Limits & Cost

1. **Quotas:** The maximum capacity (tokens-per-minute, requests-per-minute) allocated to a deployment or subscription.

2. **Rate limits:** Per-request throttling that protects shared infrastructure and controls runaway spend.

3. **Scaling decisions:** Raise capacity ahead of known traffic spikes; use serverless for unpredictable load instead of over-provisioning.

4. **Cost footprint:** Track token usage per model, per feature, and per customer to attribute and control spend.

MANAGE & MONITOR

Monitoring Model Performance, Drift & Safety Events

1. **Performance monitoring:** Track latency, error rates, and output quality trends over time, not just at launch.

2. **Model drift:** Output quality or behavior degrading over time as real-world inputs diverge from what was originally tested.

3. **Safety events:** Logged occurrences where a content filter or guardrail blocked or flagged a request or response.

4. **Exam cue:** “Quality was fine at launch but has quietly declined” is a drift-monitoring scenario.

MANAGE & MONITOR

Monitoring Data Ingestion & Search Index Health

1. **Ingestion quality:** Confirm documents are being parsed, chunked, and embedded correctly — garbage in, garbage out for RAG.

2. **Search index health:** Track indexer failures, stale content, and coverage gaps in your retrieval index.

3. **Relevance performance:** Measure whether the top retrieved results are actually the most useful ones for a given query.

4. **Exam cue:** “The agent gives outdated answers” often traces back to a stale or unhealthy search index, not the model itself.

WHAT TO WATCH

Key Monitoring Signals

1 Latency & Error Rate

How fast requests complete and how often they fail — the baseline health signals.

2 Token Usage & Cost

Consumption trends per deployment, feature, or customer segment.

3 Groundedness & Drift

Whether generated answers stay accurate and whether quality trends downward.

4 Safety Trigger Rate

How often content filters or guardrails are activated — spikes signal misuse or a bad release.

SECURITY

Securing AI Systems

1. **Managed identity:** Lets Azure resources authenticate to each other without embedding secrets or keys in code.

2. **Private networking:** Restricts a Foundry endpoint to a virtual network, blocking public internet access entirely.

3. **Keyless credentials:** Uses identity-based authentication (Microsoft Entra ID) instead of long-lived API keys.

4. **Role policies:** Role-based access control (RBAC) scopes exactly who can deploy, call, or manage a given resource.

AUTHENTICATION CHOICES

Keyless (Identity-Based) vs. Key-Based Auth

Keyless / Managed Identity

- No secret to store, rotate, or leak
- Access tied to Microsoft Entra ID identity
- Fine-grained RBAC control
- Microsoft's recommended default

API Key

- Simple to use for quick prototypes
- Must be stored, rotated, and protected
- All-or-nothing access per key
- Higher risk if leaked or hardcoded

PRACTICE CHECK 3

A security review flags that an application currently authenticates to its Foundry endpoint using a hardcoded API key. What's the recommended fix?

A. Rotate the key more frequently but keep using it

B. Switch to managed identity / keyless authentication ✓ CORRECT

C. Move the key into an environment variable only

D. Increase the deployment's rate limit

Why: Managed identity eliminates the secret entirely, removing the leak risk that hardcoded or even rotated API keys still carry — it's Microsoft's recommended default.

RESPONSIBLE AI

Responsible AI in Production Systems

1. **Why AI-103 goes further than AI-901:** AI-901 tests knowing the six principles; AI-103 tests actually implementing and instrumenting them in a running system.

2. **Where it shows up:** Every stage — model choice, prompt design, deployment, monitoring, and agent autonomy — has a responsible-AI control point.

3. **The four practical areas:** Safety filters, RAI instrumentation, auditing, and agent governance — covered next.

4. **Exam framing:** Expect “which control would you configure” questions, not “name the principle” questions.

RESPONSIBLE AI

Safety Filters, Guardrails & Content Moderation

1. **Content filters:** Screen prompts and completions for categories like violence, hate, sexual content, and self-harm, with configurable severity.

2. **Guardrails:** Broader behavioral constraints — topics the system must avoid, formats it must follow, actions it must never take.

3. **Risk detection:** Flags prompt injection attempts, jailbreak patterns, and other adversarial input before it reaches the model.

4. **Exam cue:** “Block a specific category of harmful output” is a content-filter configuration question.

RESPONSIBLE AI

RAI Instrumentation — Evaluators & Explanation Tooling

1. **Evaluators:** Automated scoring routines that measure groundedness, relevance, coherence, fluency, and safety on a test set.

2. **Safety evaluations:** Specifically probe for harmful, biased, or policy-violating output across adversarial test cases.

3. **Explanation tooling:** Surfaces why a model produced a given output — supporting the Transparency principle in a concrete, inspectable way.

4. **Exam cue:** “Prove the model doesn't hallucinate before shipping” is a groundedness-evaluator scenario.

RESPONSIBLE AI

Auditing — Trace Logging, Provenance & Approval Workflows

1. **Trace logging:** Captures the full path of a request — retrieval, tool calls, generation — for later review or debugging.

2. **Provenance metadata:** Records where content came from and how it was generated, supporting content authenticity and compliance.

3. **Approval workflows:** Requires a human sign-off before a high-stakes agent action executes.

4. **Exam cue:** “We need to reconstruct exactly what the agent did last Tuesday” is a trace-logging scenario.

RESPONSIBLE AI

Governing Agent Behavior

1. **Oversight modes:** Range from fully autonomous, to human-in-the-loop approval, to human-on-the-loop monitoring after the fact.

2. **Constraints:** Hard boundaries on what an agent is allowed to do, regardless of what a user asks for.

3. **Tool-access controls:** Restrict which tools or APIs a given agent instance is permitted to call.

4. **Exam cue:** An agent that can send emails should require approval before send — that's oversight-mode governance in action.

MEMORY BOOSTER

Recall the 4 C's of Planning a Foundry Solution

“Choose, Configure, Control, Care”

C	**Choose**	Pick the right models and Foundry services for the task.
C	**Configure**	Set up infrastructure, deployments, and CI/CD pipelines.
C	**Control**	Manage quotas, monitor health, and secure the system.
C	**Care**	Implement responsible AI — safety, evaluation, auditing, governance.

Every Domain 1 topic lives under one of these four C's — use them to sort any planning question fast.

DOMAIN 1 RECAP

The Big Picture: Plan and Manage

1. **Choosing:** Match models (LLM/SLM/multimodal/code) and Foundry services (grounding, vector search, agent tools) to the task.
2. **Setting up:** Design infrastructure, pick a deployment option, configure deployments, and wire in CI/CD.
3. **Managing:** Watch quotas, cost, performance, drift, ingestion quality, and index health — then secure it with managed identity and RBAC.
4. **Governing:** Filters, evaluators, trace logging, and oversight modes turn Responsible AI from a principle into a running control.

That's Domain 1 (25–30% of the exam) fully covered. Next: the exam's largest domain — building generative and agentic solutions.

DOMAIN TWO

Implement Generative AI and Agentic Solutions

The largest domain on the exam: building generative applications, building agents with Foundry, and optimizing generative AI systems in production.

Domain Weight: 30–35% of the exam

THE CORE OF THE EXAM

What Makes a Generative AI Application?

1. **Beyond a single prompt:** A real generative application deploys a model, grounds it in data, evaluates its output, and exposes it through a maintained SDK integration.

2. **Three building blocks ahead:** Building generative apps, building agents, and optimizing the whole system once it's live.

3. **Why this domain is 30–35%:** It's the practical core of the “developer” role AI-103 certifies — expect the deepest, most scenario-heavy questions here.

4. **Exam framing:** Questions combine multiple concepts at once — e.g., “which model, deployed which way, grounded with which retrieval method.”

Deploying & Consuming Models

1. **LLMs & small models:** Deployed the same way as in Domain 1, but here you're wiring the endpoint directly into application code.

2. **Code models:** Specialized models for generating, explaining, or reviewing code — often used in developer-tooling scenarios.

3. **Multimodal models:** Consumed when an application needs to reason over images, audio, or video alongside text in one call.

4. **Consuming a deployment:** Your app calls the deployment's endpoint with the Foundry SDK, passing a structured message list and receiving a completion.

MODEL TYPES TO DEPLOY

Matching Model Type to Application Need

1 **LLM**

Conversational apps, complex drafting, and multistep reasoning tasks.

2 **SLM**

High-volume, low-latency, narrowly scoped generation or classification.

3 **Code Model**

Code generation, code review assistants, and developer copilots.

4 **Multimodal Model**

Apps that must understand or produce images, audio, or video alongside text.

Implementing RAG in an Application

1. **Why RAG:** Grounds generated answers in real, current, private data instead of relying solely on the model's frozen training knowledge.
2. **The core loop:** Retrieve relevant content from an index, inject it into the prompt as context, then generate the response.
3. **Where retrieval lives:** Typically Azure AI Search, configured with vector, keyword, or hybrid search over your indexed content.
4. **Exam cue:** Any scenario about answering questions from private/internal data is a RAG implementation question.

THE CORE PATTERN

A RAG Pipeline, Step by Step

1

Ingest & Index

Content is chunked, embedded, and stored in a searchable index.

2

Retrieve

The user's query is matched against the index to pull relevant chunks.

3

Augment

Retrieved chunks are injected into the prompt as grounding context.

4

Generate

The model produces a response grounded in the retrieved content.

Designing Tool-Augmented & Multistep Reasoning Workflows

1. **Single-call prompting:** One request in, one response out — sufficient for simple, self-contained tasks.

2. **Tool-augmented flows:** The model can call a function or API mid-conversation to fetch data or take an action before finishing its answer.

3. **Multistep reasoning pipelines:** The task is broken into ordered stages — plan, retrieve, draft, verify — each potentially using a different model or tool.

4. **Exam cue:** A task with clear sequential stages that each need different capabilities points to a multistep pipeline, not a single prompt.

PICKING THE PATTERN

Single-Step Prompting vs. Multistep Reasoning

Single-Step Prompting

- One prompt, one response
- Simple to build and debug
- Lower latency and cost
- Best for self-contained tasks

Multistep Reasoning Pipeline

- Task broken into ordered stages
- Each stage can use a different model/tool
- Higher reliability on complex tasks
- Best for research, planning, and workflows

Evaluating Models & Apps

1. **Fabrication (hallucination) detection:** Checks whether the model asserts facts not supported by the grounding data.
2. **Relevance:** Measures whether the response actually addresses what the user asked.
3. **Quality:** Coherence, fluency, and completeness of the generated output.
4. **Safety:** Screens for harmful, biased, or policy-violating content in the response.

SCORING IT

Core Evaluation Metrics

1 Groundedness

How well the response is supported by the retrieved/grounding content, not invented.

2 Relevance

How directly the response addresses the user's actual query.

3 Coherence

Whether the response reads as logical, well-structured, and fluent.

4 Similarity / F1

How closely a generated answer matches a reference answer, when one exists.

Integrating Generative Workflows via Foundry SDKs

1. **Foundry SDKs:** Language-specific libraries (Python-first) for authenticating, deploying, calling, and evaluating models from your own code.

2. **Connectors:** Pre-built integrations to external systems — data sources, search, and enterprise apps — without hand-rolled API clients.

3. **Connecting an app to a project:** Point your app's client configuration at the Foundry project endpoint and credentials, then call deployments by name.

4. **Exam framing:** You need to know the sequence — connect, deploy target, call, evaluate — more than exact syntax.

PRACTICE CHECK 4

An application must answer questions using a company's constantly-updated internal wiki, and every answer must be traceable to a source page. What should the team implement?

A A larger context window with the whole wiki pasted into every prompt

B A RAG pipeline with an indexed, searchable wiki and grounded citations ✓ CORRECT

C Fine-tuning a model once on a wiki snapshot

D A higher temperature setting for more creative answers

Why: RAG keeps answers current (the index updates independently of the model) and traceable (retrieved chunks can be cited) — exactly what's required here.

Defining Agent Roles, Goals & Tool Schemas

1. **Role:** A clear description of what the agent is for — its persona, scope, and boundaries, written into its instructions.

2. **Goal:** The concrete outcome the agent is working toward in a given interaction, not just “be helpful.”

3. **Tool schema:** A formal definition of each tool's name, parameters, and expected output, so the model can call it correctly.

4. **Exam cue:** An agent calling a tool with the wrong parameters usually traces back to a poorly defined tool schema.

Conversation Tracking & State Management

1. **Threads:** A persistent conversation history the agent can reference across multiple turns and, often, multiple sessions.

2. **Short-term memory:** Context held for the current conversation — what the user just said.

3. **Long-term memory:** Facts or preferences persisted and recalled across separate sessions entirely.

4. **Exam cue:** “The agent should remember a customer's preference next time they chat, days later” is a long-term memory scenario.

Integrating Retrieval, Function-Calling & Memory

1. **Retrieval:** Lets the agent ground its answers in an index or knowledge store before responding.
2. **Function-calling:** Lets the agent request that your app run a specific function, then use the result to continue reasoning.
3. **Memory:** Lets the agent carry context forward instead of treating every message as a fresh conversation.
4. **Why combine them:** A production-grade agent typically needs all three working together, not any one in isolation.

WHAT AGENTS CAN CALL

Agent Tool Types

1 APIs & Custom Functions

Developer-defined functions connecting the agent to your own business logic and systems.

2 Knowledge Stores & Search

Indexed data sources the agent retrieves from to ground its responses.

3 Content Understanding

Multimodal extraction — lets an agent reason over documents, images, or audio it's given.

4 Code Interpreter

Lets the agent write and execute code for calculation, analysis, or data transformation.

Orchestrated Multi-Agent Solutions

1. **Why split into multiple agents:** Specialized agents with narrow roles are more reliable, testable, and maintainable than one agent trying to do everything.

2. **Orchestration pattern:** A coordinating agent (or workflow) routes sub-tasks to the right specialist and combines their outputs.

3. **Common topology:** Intake → specialist agents (research, drafting, review) → aggregation — mirrors a real team's division of labor.

4. **Exam cue:** A scenario with several distinct sub-tasks handled by different specialists is a multi-agent orchestration question.

SEEING IT IN ACTION

A Multi-Agent Orchestration Pattern

1

Router Agent

Classifies the incoming request and decides which specialist should handle it.

2

Specialist Agent

Executes the narrow task it's built for, using its own tools and knowledge.

3

Aggregator

Combines outputs from one or more specialists into a coherent result.

4

Reviewer / Gate

Applies policy or human-approval checks before the result is finalized.

Autonomous vs. Semiautonomous Workflows

1. **Autonomous:** The agent plans and executes a full task end-to-end without a human checkpoint mid-flow.

2. **Semiautonomous:** The agent proposes actions or drafts, but a human must approve before anything irreversible happens.

3. **Choosing between them:** Match autonomy level to the reversibility and stakes of the action — sending an email vs. drafting one.

4. **Exam cue:** "Irreversible," "financial," or "customer-facing" actions push toward semiautonomous with approval gates.

MATCHING AUTONOMY TO RISK

Autonomous vs. Semiautonomous Workflows

Autonomous

- No human checkpoint mid-task
- Fastest end-to-end execution
- Best for low-risk, reversible actions
- Requires strong upfront guardrails

Semiautonomous

- Human approval gate before key actions
- Slower, but safer for high-stakes work
- Best for irreversible or customer-facing actions
- Requires a clear, low-friction approval UI

Approval Flow Controls & Human-in-the-Loop Safeguards

1. **Approval gates:** A defined checkpoint where a human must confirm before the agent's proposed action executes.
2. **Escalation paths:** Rules for when an agent should hand off to a human rather than attempt an action itself.
3. **Confidence thresholds:** An agent below a set confidence score routes to a human instead of guessing.
4. **Exam cue:** Building the actual approval step — not just the agent's reasoning — is what this topic tests.

MEMORY BOOSTER

Recall the Agent Building Blocks

“Robots Track Meaningful Objectives”

R	**Roles & Goals**	Define what the agent is for and what outcome it's working toward.
T	**Tools**	Function-calling, APIs, and retrieval the agent can invoke.
M	**Memory**	Conversation threads and persisted context across turns and sessions.
O	**Orchestration**	Multi-agent coordination and autonomy/approval controls.

“Robots Track Meaningful Objectives” — Roles, Tools, Memory, Orchestration: the four pillars of any Foundry agent build.

PRACTICE CHECK 5

An agent is being built to autonomously issue refunds over $500 to customers. What is the most important control to add before launch?

A A larger context window

B A human approval gate before the refund executes ✓ CORRECT

C A more creative system message

D A faster, smaller model deployment

Why: High-value, irreversible financial actions call for a semiautonomous workflow with a human approval gate — autonomy should scale down as stakes and irreversibility scale up.

OPTIMIZING

Tuning Generation Behavior

1. **Prompt engineering:** Refining instructions, examples, and structure in the prompt to reliably improve output quality.

2. **Model parameters:** Temperature and top-p control creativity/randomness; max tokens controls response length and cost.

3. **Iterative tuning:** Treat this as a testing loop — change one variable, re-run evaluations, compare results.

4. **Exam cue:** “Responses are too repetitive/robotic” points to raising temperature; “responses are inconsistent” points to lowering it.

OPTIMIZING

Model Reflection & Self-Critique Loops

1. **Chain-of-thought evaluation:** Asking the model to reason step-by-step, then checking whether that reasoning holds up.

2. **Model reflection:** Having the model review and critique its own draft output before finalizing a response.

3. **Self-critique loops:** An automated cycle — generate, critique, revise — repeated until quality thresholds are met.

4. **Trade-off:** These techniques raise output quality and reliability, at the cost of extra latency and token spend.

OPTIMIZING

Observability — Tracing, Analytics & Latency

1. **Tracing:** Follows a single request through every step — retrieval, tool calls, generation — for debugging and audit.

2. **Token analytics:** Breaks down consumption by feature, user, or model to control cost and spot anomalies.

3. **Safety signals:** Rates of filter triggers, flagged content, and policy violations over time.

4. **Latency breakdowns:** Where time is actually spent — retrieval, model inference, tool calls — so you optimize the real bottleneck.

WHAT OBSERVABILITY COVERS

Signals to Instrument in Production

1 Trace Logs

Step-by-step record of exactly what a request did, end to end.

2 Token Usage

Consumption trends that drive cost and capacity planning.

3 Safety Signals

Filter trigger rates and flagged-content trends over time.

4 Latency Breakdown

Time spent per step, revealing the true bottleneck to optimize.

OPTIMIZING

Orchestrating Multiple Models & Hybrid Systems

1. **Multi-model orchestration:** Routing different parts of a task to different models — e.g., a cheap model for classification, an expensive one for final drafting.

2. **Hybrid LLM + rules engines:** Combining deterministic business rules with generative reasoning — rules handle what must be exact, the model handles what must be flexible.

3. **Why hybrid matters:** Pure LLM systems can be inconsistent on rules that must never be broken — compliance thresholds, pricing logic, eligibility checks.

4. **Exam cue:** "This calculation must always be exactly correct" points toward a deterministic rule, not a model call.

ARCHITECTURE CHOICE

Single-Model vs. Hybrid/Multi-Model Systems

Single Model

- Simpler to build and reason about
- One consistent voice and behavior
- Can be costly for simple sub-tasks
- Best for smaller, uniform workloads

Hybrid / Multi-Model

- Right-sized cost per sub-task
- Rules engines guarantee exact outcomes
- More moving parts to test and monitor
- Best for complex, high-scale production systems

MEMORY BOOSTER

Recall the Optimization Loop

“Tune, Reflect, Observe, Orchestrate”

T	**Tune**	Adjust prompts and parameters like temperature and max tokens.
R	**Reflect**	Use chain-of-thought and self-critique loops to raise reliability.
O	**Observe**	Instrument tracing, token analytics, safety, and latency signals.
O	**Orchestrate**	Route work across multiple models or hybrid rule-based systems.

Every production generative system moves through this loop continuously, not just once at launch.

PRACTICE CHECK 6

A team notices their agent's average response latency has crept up over the last month, but they aren't sure whether the cause is retrieval, the model call, or a tool. What should they check first?

A. Raise the temperature parameter

B. Review a latency breakdown from tracing/observability data ✓ CORRECT

C. Switch to a hybrid rules engine

D. Increase the max tokens setting

Why: A latency breakdown from tracing shows exactly how much time each step (retrieval, inference, tool calls) consumes, letting the team target the real bottleneck instead of guessing.

PRACTICE CHECK 7

A pricing calculation inside an agent workflow must always be mathematically exact and auditable. How should this be implemented?

A. Ask the LLM to compute it directly in the prompt

B. Lower the temperature to near zero and hope for consistency

C. Implement it as a deterministic function the agent calls as a tool ✓ CORRECT

D. Fine-tune the model specifically on pricing examples

Why: Anything that must be exact and auditable belongs in deterministic code the agent invokes as a tool — not left to a probabilistic model, regardless of temperature.

DOMAIN 2 RECAP

The Big Picture: Generative & Agentic Solutions

1. **Build:** Deploy the right model type, ground it with RAG, and design single-step or multistep workflows as the task demands.

2. **Agent-ify:** Give agents roles, tools, memory, and orchestration — matching autonomy level to the risk of each action.

3. **Operationalize:** Tune prompts and parameters, add reflection loops, instrument observability, and orchestrate hybrid systems where exactness matters.

4. **Why it's the biggest domain:** This is where planning (Domain 1) and every specialized modality (Domains 3–5) actually come together into a working product.

That's Domain 2 (30–35% of the exam) fully covered. Next: the three specialized modality domains.

DOMAIN THREE

Implement Computer Vision Solutions

Generating and editing images and video with AI, and enforcing safety and policy controls on visual content.

Domain Weight: 10–15% of the exam

A DIFFERENT LENS

Computer Vision in AI-103: Generation, Not Just Analysis

1. **The key shift from AI-901:** AI-901 tests analyzing existing images (classification, detection, OCR). AI-103 tests generating and editing new images and video.

2. **Why it matters for developers:** You're building the pipeline that turns a text prompt into finished visual content — and keeps that content safe and on-brand.

3. **Three areas ahead:** Generation, editing, and safety/policy enforcement for visual content.

4. **Exam framing:** Expect implementation questions — which workflow step, which filter, which policy control — not just definitions.

IMAGE GENERATION

Generating Images from Text Prompts & Reference Media

1. **Text-to-image:** A generative image model produces a new image from a written description.
2. **Reference-guided generation:** Supplying a reference image steers style, composition, or subject while still generating something new.
3. **Prompt specificity matters:** Detailed prompts (subject, style, lighting, composition) produce far more usable, on-brief output.
4. **Exam cue:** “Generate concept art from a written creative brief” is a text-to-image implementation scenario.

VIDEO GENERATION

Generating Videos from Text Prompts & Reference Media

1. **Text-to-video:** Generates a short video clip from a written description, extending image generation into the time dimension.

2. **Reference-guided video:** A reference image or clip can anchor the generated video's subject, motion, or style.

3. **Added complexity vs. images:** Video generation must maintain consistency — of subject, lighting, and motion — across every frame.

4. **Exam cue:** Video generation scenarios test the same prompt-and-reference pattern as images, just with temporal consistency added.

TWO GENERATION MODALITIES

Image Generation vs. Video Generation

Image Generation

- Single static frame as output
- Faster and cheaper to generate
- Use case: marketing visuals, concept art
- Key challenge: prompt fidelity

Video Generation

- Multiple frames over time as output
- Slower and more compute-intensive
- Use case: short clips, product demos
- Key challenge: frame-to-frame consistency

IMAGE EDITING

Image-Editing Workflows: Inpainting & Mask-Based Edits

1. **Inpainting:** Regenerates a specific masked region of an image while leaving the rest untouched.

2. **Mask-based edits:** A mask defines exactly which pixels are eligible for change, giving precise control over the edit area.

3. **Prompt-driven modification:** A text instruction describes the desired change within the masked region — e.g., “replace the sky with a sunset.”

4. **Exam cue:** “Change only the background, keep the product exactly the same” is a mask-based inpainting scenario.

VIDEO EDITING

Editing Generated Videos

1. **Scene or object replacement:** Swapping an element within a generated video while preserving surrounding motion and lighting.

2. **Extending or trimming:** Lengthening a clip by generating additional consistent frames, or cutting it down to the needed segment.

3. **Style adjustment:** Applying a different visual style to an already-generated video without re-generating from scratch.

4. **Exam cue:** Video-editing questions test the same edit primitives as image editing, applied across a consistent frame sequence.

PUTTING IT TOGETHER

A Prompt-Driven Image Edit Workflow

1

Select Region

Define the mask marking exactly which pixels can change.

2

Write the Edit Prompt

Describe the desired change for the masked region only.

3

Generate

The model regenerates only the masked area, blending with the rest.

4

Review & Filter

Check the result against content safety and brand policy before use.

VISUAL SAFETY

Content Filters for Unsafe or Disallowed Visual Content

1. **What gets screened:** Generated images and video are checked against categories like violence, sexual content, and hate symbolism.

2. **Applied at generation time:** Filters can block a disallowed output before it's ever returned to the user, not just after the fact.

3. **Severity configuration:** Like text filters, visual filters support configurable strictness based on audience and use case.

4. **Exam cue:** Visual content safety uses the same filter-configuration pattern as text — just applied to image/video output.

VISUAL SAFETY

Indirect Prompt Injection via Images

1. **The attack:** Malicious instructions are hidden as text embedded within an image, aimed at hijacking a model that processes that image.

2. **Why it's dangerous:** A multimodal model reading the image may follow the embedded instructions as if they came from the legitimate user.

3. **Mitigation:** Scan embedded text in uploaded images, and treat any instructions found there as untrusted content rather than a command.

4. **Exam cue:** A model behaving unexpectedly after processing an uploaded image should raise indirect prompt injection as a suspect.

POLICY ENFORCEMENT

Enforcing Visual Policy Rules

1 Watermarking

Marks generated content as AI-produced for transparency and provenance tracking.

2 Prohibited Symbols

Flags generated content containing symbols or imagery the organization disallows.

3 Brand Usage Requirements

Ensures generated visuals follow brand color, logo, and style guidelines.

4 Inappropriate Content Detection

Catches content that violates policy even if it passes baseline safety filters.

MEMORY BOOSTER

Recall the Computer Vision Workflow

“Great Editors Filter Everything”

G	**Generate**	Create images and video from text prompts and reference media.
E	**Edit**	Apply inpainting, mask-based edits, and video edits precisely.
F	**Filter**	Block unsafe or disallowed visual content at generation time.
E	**Enforce**	Apply watermarks, brand rules, and policy checks before release.

“Great Editors Filter Everything” — the full generation-to-policy pipeline for Domain 3.

PRACTICE CHECK 8

A multimodal agent processes a user-uploaded image and then unexpectedly starts giving instructions unrelated to what the user asked. What should the team investigate first?

A. Whether the temperature parameter is too high

B. Whether the image contains hidden text causing indirect prompt injection ✓ CORRECT

C. Whether the max tokens limit was exceeded

D. Whether the wrong deployment region was selected

Why: Unexpected behavior right after processing an uploaded image is the classic signature of indirect prompt injection via embedded text — it should be the first thing investigated.

DOMAIN FOUR

Implement Text Analysis Solutions

LLM-powered text analysis and translation, plus speech solutions that give agents a voice and ears.

Domain Weight: 10–15% of the exam

A DIFFERENT LENS

Text Analysis in AI-103: LLM-Powered, Not Just Prebuilt APIs

1. **The key shift from AI-901:** AI-901 tests recognizing prebuilt NLP tasks; AI-103 tests implementing them using generative prompting and Foundry Tools.

2. **Why prompting instead of fixed APIs:** A single well-prompted model can flexibly extract, classify, and reformat text — adapting to domain-specific needs a fixed API can't.

3. **Two areas ahead:** LLM-powered text analysis, and speech solutions built for agentic interaction.

4. **Exam framing:** Expect “which prompt/service pattern would you build” questions rather than “name the task” questions.

TEXT ANALYSIS

Extracting Entities, Topics, Summaries & Structured JSON

1. **Prompted extraction:** A generative prompt instructs the model to pull entities, topics, or a summary directly from input text.

2. **Structured JSON output:** Prompts (or structured-output features) constrain the model to return a specific schema your app can parse reliably.

3. **Foundry Tools support:** Pre-built tools can handle common extraction patterns without hand-writing every prompt from scratch.

4. **Exam cue:** “Return the result as valid JSON matching this schema” is a structured-output implementation question.

TEXT ANALYSIS

Detecting Sentiment, Tone, Safety Issues & Sensitive Content

1. **Sentiment & tone detection:** Prompted classification of emotional valence and communication style in a passage.

2. **Safety issue detection:** Flags harmful, threatening, or policy-violating language in user or generated text.

3. **Sensitive content detection:** Identifies PII or other sensitive data categories that may need masking or special handling.

4. **Exam cue:** “Flag messages that might contain personal data before they're logged” is a sensitive-content detection scenario.

TRANSLATION

Translation Solutions

1. **Azure Translator (in Foundry Tools):** A dedicated, purpose-built translation service — fast, consistent, and cost-efficient at scale.

2. **LLM-powered translation:** A generative model translates as part of a larger flow, useful when translation needs to be tone-aware or context-sensitive.

3. **Choosing between them:** Use Azure Translator for high-volume, straightforward translation; use LLM-powered translation when nuance or integration with other reasoning matters.

4. **Exam cue:** “Translate while preserving a specific brand voice” points to LLM-powered translation, not the standalone Translator service.

TWO TRANSLATION PATHS

Azure Translator vs. LLM-Powered Translation

Azure Translator

- Purpose-built, highly optimized service
- Fast and cost-efficient at high volume
- Consistent, literal translation quality
- Best for straightforward bulk translation

LLM-Powered Translation

- Context- and tone-aware translation
- Can integrate with broader reasoning flows
- More flexible for nuance and style
- Best when translation is part of a bigger task

TEXT ANALYSIS

Customizing LLM Outputs for Domain Tasks

1. **Compliance summarization:** Prompting the model to summarize documents specifically against regulatory or policy criteria.

2. **Domain extraction:** Tailoring extraction prompts to a specific industry's vocabulary — legal clauses, medical terms, financial figures.

3. **Why customization matters:** A generic summarization prompt misses domain-specific nuance that a tailored one captures reliably.

4. **Exam cue:** “Summarize contracts specifically flagging non-standard clauses” is a domain-customization scenario.

SPEECH

Speech-to-Text & Text-to-Speech for Agentic Interactions

1. **Speech-to-text:** Converts a user's spoken input into text the agent can reason over, in real time or from a recording.

2. **Text-to-speech:** Converts the agent's generated response into natural spoken audio for voice interfaces.

3. **Why “agentic” matters here:** These conversions must integrate smoothly into an agent's turn-taking flow, not just work as standalone transcription.

4. **Exam cue:** A phone-based support agent needs both directions wired into its conversation loop.

SPEECH

Speech as an Agent Modality & Custom Speech Models

1. **Speech as a modality:** Voice becomes just another input/output channel an agent supports, alongside text and images.

2. **Custom speech models:** Trained or adapted to a specific domain's vocabulary, accents, or acoustic environment for higher accuracy.

3. **Custom voice:** A distinctive, brand-specific synthetic voice trained from voice samples for text-to-speech output.

4. **Exam cue:** Poor recognition accuracy on industry jargon points toward a custom speech model, not a different base LLM.

SPEECH

Multimodal Reasoning from Audio & Speech Translation

1. **Multimodal audio reasoning:** A model reasons directly over audio input — tone, emphasis, background sound — not just its transcribed text.

2. **Speech translation:** Converts spoken audio in one language directly into text or speech in another.

3. **Combining speech and language models:** Foundry Tools let you chain speech capabilities with LLM reasoning in one workflow.

4. **Exam cue:** “Detect the caller's frustration from their tone of voice” is multimodal audio reasoning, not plain transcription.

QUICK REFERENCE

Text & Speech Capabilities Recap

1 Prompted Extraction

Entities, topics, summaries, and structured JSON via generative prompting.

2 Detection

Sentiment, tone, safety issues, and sensitive content flagged automatically.

3 Translation

Azure Translator for volume, LLM-powered translation for nuance.

4 Speech I/O

Speech-to-text and text-to-speech wired into an agent's conversation loop.

MEMORY BOOSTER

Recall the Text & Speech Workflow

“Every Detective Talks Smoothly”

E	**Extract**	Pull entities, topics, summaries, and structured JSON from text.
D	**Detect**	Flag sentiment, tone, safety issues, and sensitive content.
T	**Translate**	Use Azure Translator or LLM-powered translation flows.
S	**Speak**	Convert speech to text and text to speech for agentic interaction.

“Every Detective Talks Smoothly” — the four moves of Domain 4, in order.

PRACTICE CHECK 9

A voice-based support agent frequently misrecognizes specialized industry terminology from callers. What's the most effective fix?

A. Increase the model's max tokens

B. Train or adapt a custom speech model for the domain vocabulary ✓ CORRECT

C. Switch from text-to-speech to speech-to-text only

D. Lower the temperature parameter

Why: Recognition accuracy on specialized or unusual vocabulary is exactly what custom speech models are built to improve, by adapting to domain-specific terms and acoustics.

DOMAIN FIVE

Implement Information Extraction Solutions

Building the retrieval and grounding pipelines that power RAG and agentic knowledge access — ingestion, indexing, search, and enrichment.

Domain Weight: 10–15% of the exam

THE FINAL PIECE

Information Extraction: Building Retrieval & Grounding Pipelines

1. **Why this domain exists:** Every RAG scenario in Domain 2 depends on a well-built retrieval pipeline — this domain tests how you actually build one.

2. **The pipeline in one sentence:** Ingest raw content, enrich and index it, then make it searchable so agents and apps can ground their answers in it.

3. **Five areas ahead:** Ingestion, search methods, enrichment, RAG ingestion with OCR, and connecting pipelines to workflows.

4. **Exam framing:** Expect questions about which stage of the pipeline is misconfigured when an agent's answers are wrong or outdated.

INGESTION

Ingesting & Indexing Multimodal Content

1. **Documents:** PDFs, Word files, and web pages parsed into chunks suitable for embedding and retrieval.

2. **Images:** Indexed using vision-derived captions or embeddings so they're findable by meaning, not just filename.

3. **Audio & video:** Transcribed and/or indexed via extracted metadata so spoken or visual content becomes searchable.

4. **Exam cue:** “Users should be able to search across PDFs, screenshots, and recorded calls in one place” is a multimodal ingestion scenario.

SEARCH METHODS

Semantic, Hybrid & Vector Search for Grounding

1. **Semantic search:** Ranks results by conceptual relevance to the query's meaning, not just keyword overlap.

2. **Vector search:** Compares query and content embeddings directly by similarity — the mechanical foundation of semantic matching.

3. **Hybrid search:** Combines vector similarity with keyword matching, then re-ranks — typically the strongest default for RAG grounding.

4. **Exam cue:** “Must match exact product codes AND handle natural-language questions” is a hybrid-search scenario.

CHOOSING A SEARCH METHOD

Vector Search vs. Hybrid Search

Vector (Semantic) Search

- Matches by meaning via embeddings
- Great for natural-language questions
- Can miss exact codes/IDs/keywords
- Best for conceptual, exploratory queries

Hybrid Search

- Combines keyword + vector, then re-ranks
- Handles exact terms and paraphrased queries
- Slightly more complex to configure
- Best general-purpose default for RAG

ENRICHMENT

Enrichment — Custom & Built-In Skills

1. **Built-in skills:** Pre-built enrichment steps — OCR, entity recognition, image captioning — applied automatically during indexing.

2. **Custom skills:** Your own enrichment logic, plugged into the pipeline for domain-specific processing built-ins don't cover.

3. **Text enrichment:** Extracting entities, key phrases, or language metadata to improve searchability.

4. **Layout enrichment:** Preserving document structure — headings, tables, reading order — so retrieved chunks stay coherent.

RAG INGESTION

Configuring RAG Ingestion Flow with OCR

1. **The problem OCR solves:** Scanned documents and images-of-text have no extractable text layer until OCR runs first.

2. **Where OCR fits:** It's typically the first enrichment step — text must be extracted before it can be chunked, embedded, and indexed.

3. **Chunking after OCR:** Extracted text is split into appropriately sized chunks that balance context completeness against retrieval precision.

4. **Exam cue:** “Scanned PDF invoices aren't showing up in search results” points to a missing or misconfigured OCR ingestion step.

END TO END

The Complete Grounding Pipeline

1

Ingest

Pull in documents, images, audio, and video from source systems.

2

Enrich

Run OCR and built-in/custom skills to extract text, entities, and layout.

3

Index

Chunk and embed content into a searchable, hybrid-ready index.

4

Ground

Retrieved content is injected into prompts for RAG and agent tools.

CONNECTING IT

Connecting Retrieval Pipelines to Workflows & Agent Tools

1. **As a RAG data source:** The index feeds directly into a generative app's retrieve-then-generate pattern.
2. **As an agent tool:** Wrapped as a callable “knowledge” tool an agent invokes mid-conversation when it needs grounding.
3. **As a workflow step:** Embedded as one stage in a larger multistep reasoning pipeline, feeding downstream stages.
4. **Exam cue:** The same underlying index can serve a chatbot, an agent's file-search tool, and a batch workflow — know all three integration points.

CORE VOCABULARY

Azure AI Search Core Concepts

1 Index

The searchable structure holding your processed, chunked, and embedded content.

2 Skillset

The ordered set of enrichment steps (OCR, entity extraction) applied during indexing.

3 Indexer

The process that pulls source content through the skillset and into the index.

4 Vector Store

The component storing embeddings for similarity-based retrieval.

MEMORY BOOSTER

Recall the Retrieval & Grounding Pipeline

“Island Ideas Search Gold”

I	**Ingest**	Pull in documents, images, audio, and video from source systems.
I	**Index**	Chunk, embed, and structure content with semantic, hybrid, or vector search.
S	**Search**	Retrieve the most relevant content for a given query.
G	**Ground**	Connect retrieved content into RAG apps and agent tools.

“Island Ideas Search Gold” — Ingest, Index, Search, Ground: the pipeline behind every RAG and agent knowledge feature.

PRACTICE CHECK 10

An agent's answers reference outdated policy details even though the source documents were updated last week. Where should the team look first?

A The model's temperature setting

B Whether the search index has been re-ingested/re-indexed since the update ✓ CORRECT

C The agent's system message wording

D The deployment's region setting

Why: RAG answers are only as current as the index. If source documents changed but the index wasn't refreshed, the agent will keep retrieving and grounding on stale content.

DOMAIN 5 RECAP

The Big Picture: Information Extraction

1. **Ingest broadly:** Documents, images, audio, and video all become candidate content for grounding.
2. **Enrich deliberately:** OCR and built-in/custom skills turn raw content into structured, searchable text and metadata.
3. **Search smartly:** Hybrid search is usually the strongest default, blending exact-match and semantic relevance.
4. **Connect everywhere:** The same index powers RAG apps, agent knowledge tools, and multistep workflows alike.

That's all five domains covered. Next: locking it all in for exam day.

MEMORY BOOSTER

The Master Map of AI-103

“Please Give Cats Tasty Ice Cream”

P	**Plan & Manage**	Domain 1 (25–30%) — choosing services, setup, security, responsible AI.
G	**Generative & Agentic**	Domain 2 (30–35%) — building apps, agents, and optimizing them in production.
C	**Computer Vision**	Domain 3 (10–15%) — image and video generation, editing, and visual safety.
T	**Text Analysis**	Domain 4 (10–15%) — LLM-powered text analysis and speech solutions.
I	**Information Extraction**	Domain 5 (10–15%) — retrieval and grounding pipelines for RAG and agents.

“Please Give Cats Tasty Ice Cream” — P-G-C-T-I, in the exact order Microsoft lists the domains.

FLASHCARD WALL 1

Vocabulary Rapid Review — Foundry & Agents

1 Grounding

Supplying a model with real, trustworthy data at prompt time to improve factual accuracy.

2 Tool Schema

The formal definition of a tool's name, parameters, and output so an agent can call it correctly.

3 Managed Identity

Keyless, identity-based authentication between Azure resources — no secrets to leak.

4 Groundedness (metric)

How well a generated response is supported by retrieved content, versus invented.

5 Approval Gate

A checkpoint requiring human sign-off before an agent's action executes.

6 Trace Logging

A step-by-step record of a request's full path — retrieval, tool calls, generation.

FLASHCARD WALL 2

Vocabulary Rapid Review — Vision, Text & Extraction

1 Inpainting

Regenerating a masked region of an image while leaving the rest unchanged.

2 Indirect Prompt Injection

Malicious instructions hidden as text embedded within an uploaded image.

3 Structured JSON Output

Constraining a model's response to a defined schema your app can parse reliably.

4 Hybrid Search

Combining keyword and vector search, then re-ranking — the strongest RAG default.

5 Skillset

The ordered set of enrichment steps (OCR, entity extraction) run during indexing.

6 Custom Speech Model

A speech model adapted to domain vocabulary, accents, or acoustic conditions.

PRACTICE CHECK 11

A generative app must never let the model perform an exact tax calculation itself, but must let it explain the result conversationally. What's the right architecture?

A A single LLM call handling both the math and the explanation

B A deterministic function tool for the calculation, called by the model, which then explains the result ✓ CORRECT

C A higher temperature setting for more careful math

D Fine-tuning the model specifically on tax examples

Why: Exact, auditable calculations belong in a deterministic tool the model calls — a hybrid pattern where the model handles language and the tool handles precision.

PRACTICE CHECK 12

An organization wants generated marketing images to always carry a mark indicating they were AI-produced. Which control should they configure?

A A higher image-generation resolution setting

B Watermarking as part of visual policy enforcement ✓ CORRECT

C A stricter content filter severity level

D Reference-guided generation instead of text-to-image

Why: Watermarking is the specific policy control for marking generated content as AI-produced, supporting transparency and provenance.

PRACTICE CHECK 13

A team wants their agent's prompt changes to automatically trigger a full evaluation suite before being promoted to production. What should they implement?

A. A manual weekly spot-check process

B. A CI/CD pipeline integrated with the Foundry project ✓ CORRECT

C. A higher rate limit on the deployment

D. A second, redundant model deployment

Why: Automatically running evaluations on every change and gating promotion is exactly what a CI/CD pipeline integrated with Foundry is designed to do.

AVOID THESE TRAPS

Common Exam Traps & Tricky Distinctions

1. **Autonomous vs. Semiautonomous:** Autonomous has no mid-task human checkpoint; semiautonomous requires approval before key actions.

2. **Vector vs. Hybrid Search:** Vector search matches by meaning alone; hybrid adds keyword matching for exact terms and codes.

3. **Model Drift vs. a Bad Deployment:** Drift is gradual degradation over time; a bad deployment is an immediate regression right after a change.

4. **Managed Identity vs. API Key:** Managed identity is keyless and identity-based; an API key is a secret that must be stored and rotated.

5. **RAG vs. Fine-Tuning:** RAG grounds answers in fresh external data at query time; fine-tuning permanently changes the model's weights.

KNOW THE HISTORY

AI-102 vs. AI-103: What Changed

AI-102 (Retiring June 2026)

- Focused on Azure AI services individually
- Centered on Azure AI Search, Azure OpenAI
- Python or C# experience expected
- Generative AI as one topic among many

AI-103 (Current Exam)

- Centered on Microsoft Foundry end-to-end
- Deep focus on agents, RAG, and orchestration
- Python app-development experience expected
- Generative & agentic AI as the dominant focus

GAME DAY

Your Exam Day Plan

1

Sleep & Review

Skim mnemonic slides the night before — don't cram brand-new material.

2

Arrive Early

Check in with ID at least 15 minutes ahead of your Pearson VUE appointment.

3

Triage Scenarios

Answer confident questions first; flag multi-concept scenarios to revisit.

4

Find the Trade-off

Most scenario questions hinge on one trade-off — cost, risk, or latency. Name it first.

FINAL TIPS

Last-Mile Study Advice

1. **Build something small in Foundry:** A real RAG app or a two-tool agent cements far more than reading alone — this exam rewards hands-on time.

2. **Say the mnemonics aloud:** Verbalizing — not just reading — dramatically improves recall under exam pressure.

3. **Re-take the practice checks cold:** Do them again 24 hours later without looking at the explanations first.

4. **Weight your review to Domains 1 and 2:** Together they're 55–65% of the exam — make sure Foundry setup and agent-building are rock solid.

ONE LAST LOOK

The Numbers That Matter

30–35%

Generative & Agentic

The single largest domain — study it hardest

25–30%

Plan & Manage

Foundry setup, security, and responsible AI

100 min

Exam Duration

Longer than AI-901 — pace yourself accordingly

700/1000

Passing Score

Your target scaled score on exam day

MEMORY BOOSTER

The Complete Memory Wall

Every Mnemonic From This Guide, In One Place

1	**Foundry Setup Chain**	“Design → Deploy → Configure → Connect”.
2	**4 C's of Planning**	“Choose, Configure, Control, Care” — Domain 1's four pillars.
3	**Agent Building Blocks**	“Robots Track Meaningful Objectives” — Roles, Tools, Memory, Orchestration.
4	**Optimization Loop**	“Tune, Reflect, Observe, Orchestrate.”
5	**Computer Vision Workflow**	“Great Editors Filter Everything” — Generate, Edit, Filter, Enforce.
6	**Text & Speech Workflow**	“Every Detective Talks Smoothly” — Extract, Detect, Translate, Speak.
7	**Retrieval & Grounding**	“Island Ideas Search Gold” — Ingest, Index, Search, Ground.
8	**The Five Domains**	“Please Give Cats Tasty Ice Cream” — Plan, Generative, Vision, Text, Extraction.

Eight phrases. Every major checklist on the exam. Review this one slide the morning of your test.

YOU'RE READY

You're Ready for AI-103.

Foundry planned. Agents built. Modalities covered. Mnemonics locked in.

- Revisit the gold mnemonic slides one more time before test day.
- Get hands-on in the real Microsoft Foundry portal if you haven't already.
- On scenario questions, find the underlying trade-off first, then choose.
- After passing: explore advanced agent-orchestration and Azure AI Search specializations next.

YOU'RE READY

Good Luck — Go Earn That Certification.

Microsoft Certified: Azure AI App and Agent Developer Associate

- This guide covered all five exam domains in full, weighted to match the real blueprint.
- Keep this deck as your final-review companion in the days before your exam.
- Official exam updates: learn.microsoft.com/certifications/azure-ai-app-and-agent-developer-associate
- Thank you for studying with this guide — now go build (and pass) with confidence.